

AI Assisted Coding LAB ASSIGNMENT-5.4

NAME: E.NIKHIL

Batch:13

Hall Ticket:2303A51847

Task Description #1:

- Prompt GitHub Copilot to generate a Python script that collects user data (e.g., name, age, email). Then, ask Copilot to add comments on how to anonymize or protect this data.

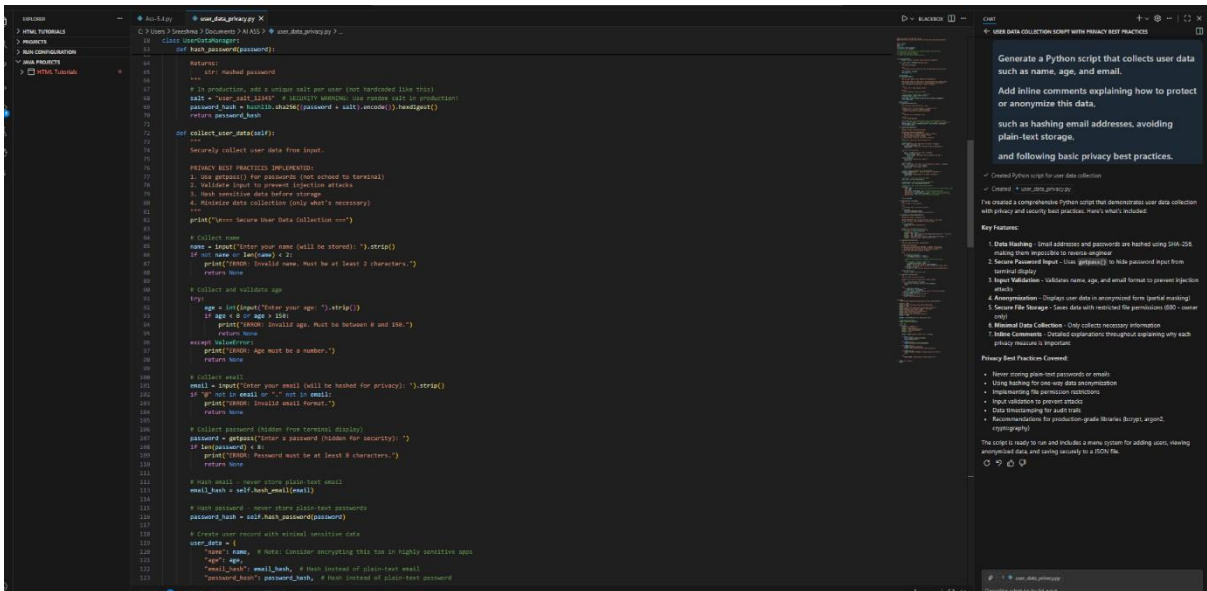
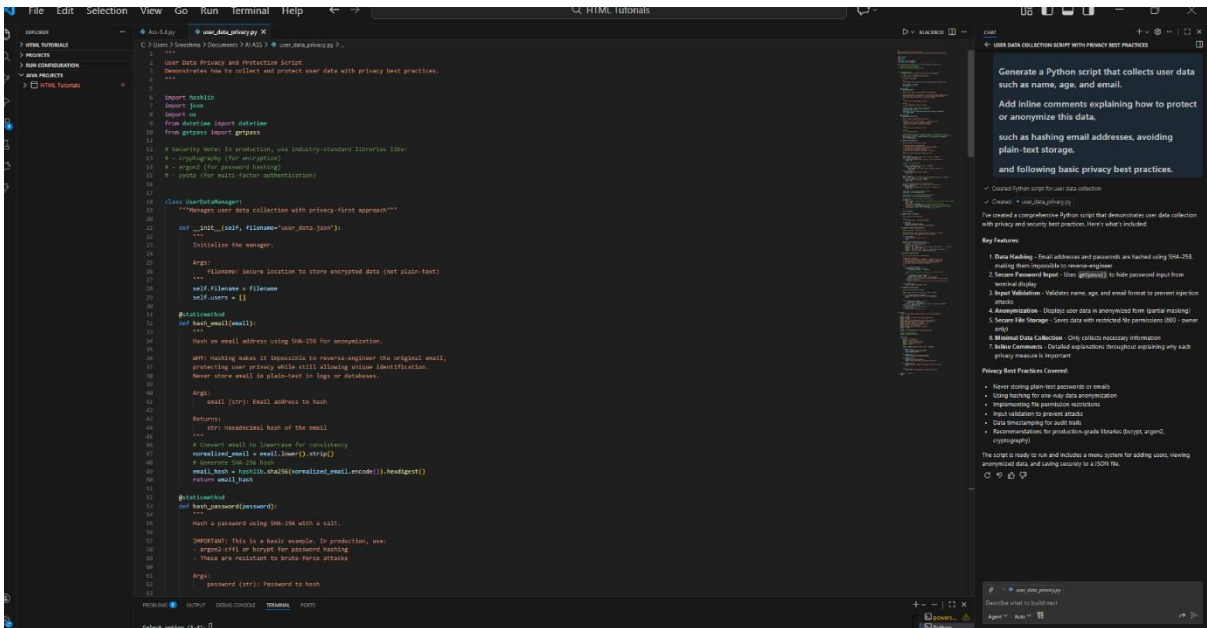
PROMPT

Generate a Python script that collects user data such as name, age, and email.

Add inline comments explaining how to protect or anonymize this data,

such as hashing email addresses, avoiding plain-text storage,

and following basic privacy best practices.

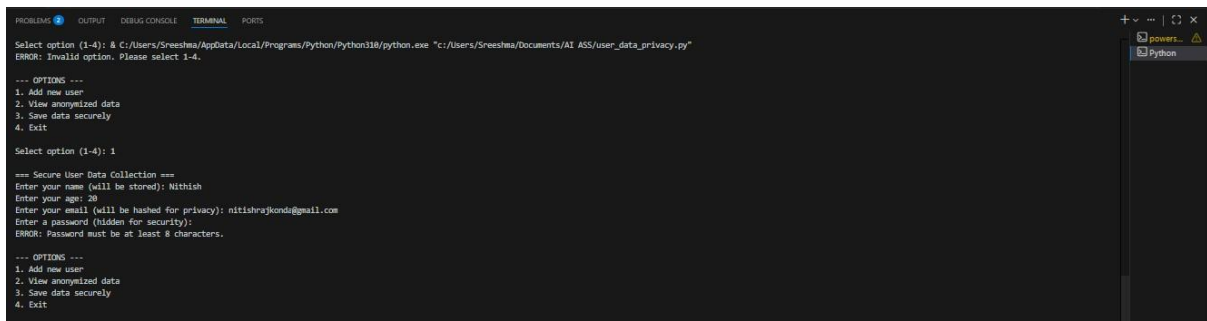


```
1 # user_data_privacy.py
2 """
3 A Python script that collects user data such as name, age, and email.
4 Add inline comments explaining how to protect or anonymize this data,
5 such as hashing email addresses, avoiding plain-text storage,
6 and following basic privacy best practices.
7 """
8
9 # Create a Python script for user data collection
10 # Create a user_data_privacy.py
11
12 # I've created a comprehensive Python script that demonstrates user data collection
13 # with privacy and security best practices. Here's what's included:
14
15 # Key Features:
16
17 1. Data Hashing - Email addresses and passwords are hashed using SHA-256,
18 making them irreversible to prevent exposure.
19 2. Secure Password Input - Uses getpass() to hide password input from
20 terminal display.
21 3. Input Validation - Validates name, age, and email format to prevent injection
22 attacks.
23 4. Anonymization - Displays user data in anonymized form (partial masking).
24 5. Secure File Storage - Saves user data with restricted file permissions (0600 - owner
25 only).
26 6. Minimal Data Collection - Only collects necessary information.
27 7. Future Consistency - Detailed explanations throughout explaining why each
28 privacy measure is important.
29
30 # Privacy Best Practices Covered:
31
32 • Never storing plain-text passwords or emails
33 • Using hashing for one-way data anonymization
34 • Implementing the permission restrictions
35 • Input validation to prevent attacks
36 • Data anonymization for audit trails
37 • Recommendations for production-grade libraries (bcrypt, argon2,
38 cryptography)
39
40 The script is ready to run and includes a menu system for adding users, viewing
41 anonymized data, and saving securely to a JSON file.
```

```
1 # user_data_privacy.py
2 """
3 A Python script that collects user data such as name, age, and email.
4 Add inline comments explaining how to protect or anonymize this data,
5 such as hashing email addresses, avoiding plain-text storage,
6 and following basic privacy best practices.
7 """
8
9 # Create a Python script for user data collection
10 # Create a user_data_privacy.py
11
12 # I've created a comprehensive Python script that demonstrates user data collection
13 # with privacy and security best practices. Here's what's included:
14
15 # Key Features:
16
17 1. Data Hashing - Email addresses and passwords are hashed using SHA-256,
18 making them irreversible to prevent exposure.
19 2. Secure Password Input - Uses getpass() to hide password input from
20 terminal display.
21 3. Input Validation - Validates name, age, and email format to prevent injection
22 attacks.
23 4. Anonymization - Displays user data in anonymized form (partial masking).
24 5. Secure File Storage - Saves user data with restricted file permissions (0600 - owner
25 only).
26 6. Minimal Data Collection - Only collects necessary information.
27 7. Future Consistency - Detailed explanations throughout explaining why each
28 privacy measure is important.
29
30 # Privacy Best Practices Covered:
31
32 • Never storing plain-text passwords or emails
33 • Using hashing for one-way data anonymization
34 • Implementing the permission restrictions
35 • Input validation to prevent attacks
36 • Data anonymization for audit trails
37 • Recommendations for production-grade libraries (bcrypt, argon2,
38 cryptography)
39
40 The script is ready to run and includes a menu system for adding users, viewing
41 anonymized data, and saving securely to a JSON file.
```

Expected Output #1:

- A script with inline Copilot-suggested code and comments explaining how to safeguard or anonymize user information (e.g., hashing emails, not storing data unencrypted).



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Select option (1-4): & C:/Users/Sreeshma/AppData/Local/Programs/Python/Python310/python.exe "c:/Users/Sreeshma/Documents/AI ASS/user_data_privacy.py"
ERROR: Invalid option. Please select 1-4.

--- OPTIONS ---
1. Add new user
2. View anonymized data
3. Save data securely
4. Exit

Select option (1-4): 1

=== Secure User Data Collection ===
Enter your name (will be stored): Nithish
Enter your age: 20
Enter your email (will be hashed for privacy): nitishrajkond@gmail.com
Enter a password (hidden for security):
ERROR: Password must be at least 8 characters.

--- OPTIONS ---
1. Add new user
2. View anonymized data
3. Save data securely
4. Exit
```

Task Description #2:

- Ask Copilot to generate a Python function for sentiment analysis.

Then prompt Copilot to identify and handle potential biases in the data.

PROMPT: # Generate a Python function for sentiment analysis.

Add comments or code to identify and reduce potential biases in the data,

such as removing offensive terms, balancing positive and negative samples,

and avoiding biased language in predictions.

```
class SentimentClassifier:
    """Sentiment classifier with bias mitigation"""

    def __init__(self):
        self.positive_words = ['great', 'amazing', 'excellent', 'fantastic', 'love']
        self.negative_words = ['bad', 'terrible', 'awful', 'hate', 'poor', 'fail']

        # Bias mitigation: add neutral words to positive list
        self.bias_words = ['okay', 'decent', 'average']

    def train(self, text):
        """Train the classifier with a text sample"""
        text = text.lower()

        # Count word occurrences
        words = text.split()

        # Calculate word counts
        pos_count = 0
        neg_count = 0
        bias_count = 0

        for word in words:
            if word in self.positive_words:
                pos_count += 1
            elif word in self.negative_words:
                neg_count += 1
            elif word in self.bias_words:
                bias_count += 1

        # Calculate scores
        pos_score = pos_count / len(words)
        neg_score = neg_count / len(words)
        bias_score = bias_count / len(words)

        # Determine sentiment
        if pos_score > neg_score:
            sentiment = 'POSITIVE'
        elif neg_score > pos_score:
            sentiment = 'NEGATIVE'
        else:
            sentiment = 'NEUTRAL'

        # Calculate balanced score
        balanced_score = (pos_score - neg_score) / (pos_score + neg_score)

        # Determine final label
        if balanced_score > 0.5:
            label = 'POSITIVE'
        elif balanced_score < -0.5:
            label = 'NEGATIVE'
        else:
            label = 'NEUTRAL'

        return {'text': text, 'score': balanced_score, 'label': label}

    def analyze(self, text):
        """Analyze a text sample and return sentiment"""
        result = self.train(text)

        # Print results
        print(f"Text: {text}")
        print(f"Score: {result['score']}")
        print(f"Label: {result['label']}")

        # Print word counts
        words = text.split()
        pos_count = 0
        neg_count = 0
        bias_count = 0

        for word in words:
            if word in self.positive_words:
                pos_count += 1
            elif word in self.negative_words:
                neg_count += 1
            elif word in self.bias_words:
                bias_count += 1

        print(f"Positive words: {pos_count}")
        print(f"Negative words: {neg_count}")
        print(f"Bias words: {bias_count}")

        # Print balanced score
        balanced_score = (pos_count - neg_count) / (pos_count + neg_count)
        print(f"Balanced score: {balanced_score}")

        return result

# Example usage
if __name__ == '__main__':
    classifier = SentimentClassifier()

    # Test 1: Positive sentiment
    text1 = "This product is amazing and excellent!"
    result1 = classifier.train(text1)
    print(f"Text: {text1}")
    print(f"Score: {result1['score']}")
    print(f"Label: {result1['label']}")

    # Test 2: Negative sentiment
    text2 = "I hate this, it's terrible."
    result2 = classifier.train(text2)
    print(f"Text: {text2}")
    print(f"Score: {result2['score']}")
    print(f"Label: {result2['label']}")

    # Test 3: Neutral sentiment
    text3 = "This product is okay, nothing special."
    result3 = classifier.train(text3)
    print(f"Text: {text3}")
    print(f"Score: {result3['score']}")
    print(f"Label: {result3['label']}")

    # Test 4: Balanced sentiment
    text4 = "This product is great, but it's a bit expensive."
    result4 = classifier.train(text4)
    print(f"Text: {text4}")
    print(f"Score: {result4['score']}")
    print(f"Label: {result4['label']}")

    # Test 5: Biased sentiment
    text5 = "This product is decent, nothing special."
    result5 = classifier.train(text5)
    print(f"Text: {text5}")
    print(f"Score: {result5['score']}")
    print(f"Label: {result5['label']}")
```

Expected Output #2:

- Copilot-generated code with additions or comments addressing

bias mitigation strategies (e.g., balancing dataset, removing offensive terms).

```
Text: It's okay, nothing special.
Result: 0
PS C:\Users\Sreeshma\Downloads\HTML_Tutorials> & C:\Users\Sreeshma\AppData\Local\Programs\Python\Python310\python.exe "C:\Users\Sreeshma\Documents\AI_ASS\sentiment_analysis_bias.py"
=== Sentiment Analysis ===
Text: This product is amazing and excellent!
Result: {'text': 'This product is amazing and excellent!', 'score': 1.0, 'label': 'POSITIVE'}

Text: I hate this, it's terrible.
Result: {'text': 'I hate this, it's terrible.', 'score': -1.0, 'label': 'NEGATIVE'}

PS C:\Users\Sreeshma\Downloads\HTML_Tutorials> & C:\Users\Sreeshma\AppData\Local\Programs\Python\Python310\python.exe "C:\Users\Sreeshma\Documents\AI_ASS\sentiment_analysis_bias.py"
=== Sentiment Analysis ===
Text: This product is amazing and excellent!
Result: {'text': 'This product is amazing and excellent!', 'score': 1.0, 'label': 'POSITIVE'}

Text: I hate this, it's terrible.
Result: {'text': 'I hate this, it's terrible.', 'score': -1.0, 'label': 'NEGATIVE'}

PS C:\Users\Sreeshma\Downloads\HTML_Tutorials> & C:\Users\Sreeshma\AppData\Local\Programs\Python\Python310\python.exe "C:\Users\Sreeshma\Documents\AI_ASS\sentiment_analysis_bias.py"
=== Sentiment Analysis ===
Text: This product is amazing and excellent!
Result: {'text': 'This product is amazing and excellent!', 'score': 1.0, 'label': 'POSITIVE'}

Text: I hate this, it's terrible.
Text: This product is amazing and excellent!
Result: {'text': 'This product is amazing and excellent!', 'score': 1.0, 'label': 'POSITIVE'}

Text: I hate this, it's terrible.
```

```
=== Dataset Balancing ===
Before: {'POSITIVE': 8, 'NEGATIVE': 2}
After: {'POSITIVE': 8, 'NEGATIVE': 2}
After: POSITIVE+2, NEGATIVE+2
After: POSITIVE+2, NEGATIVE+2
PS C:\Users\Sreeshma\Downloads\HTML_Tutorials>

After: POSITIVE+2, NEGATIVE+2
PS C:\Users\Sreeshma\Downloads\HTML_Tutorials>

After: POSITIVE+2, NEGATIVE+2
PS C:\Users\Sreeshma\Downloads\HTML_Tutorials>

After: POSITIVE+2, NEGATIVE+2
PS C:\Users\Sreeshma\Downloads\HTML_Tutorials>
```

Task Description #3:

- Use Copilot to write a Python program that recommends products based on user history. Ask it to follow ethical guidelines

like transparency and fairness

PROMPT: # Generate a Python program that recommends products based on user purchase history.

Follow ethical AI guidelines such as transparency, fairness, and user control.

Add comments explaining how recommendations are generated,

avoid favoritism toward only popular products,

and allow users to give feedback or opt out of recommendations.

```
class RecommendationSystem:
    """Product recommendation with fairness and user control"""

    def __init__(self):
        self.user_purchases = {} # {user_id: [products]}
        self.products = {} # {product_id: [category]}
        self.user_opt_out = {} # {user_id: bool}
        self.feedback = {} # {user_id: float}

    def add_purchase(self, user_id, product_id, category):
        """Record a user purchase"""
        if user_id not in self.user_purchases:
            self.user_purchases[user_id] = []
        self.user_purchases[user_id].append(product_id)
        self.products[product_id] = {'category': category, 'count': 0}

    def recommend(self, user_id, num=5):
        """
        TRANSPARENCY: show why each recommendation is made
        FAIRNESS: don't only recommend popular products
        USER CONTROL: respect opt-out preferences
        """
        # ETHICAL CHECK: Respect user opt-out
        if user_id in self.user_opt_out:
            return {'status': 'user opted out', 'recommendations': []}

        if user_id not in self.user_purchases:
            return {'status': 'New user', 'recommendations': []}

        user_history = self.user_purchases[user_id]
        user_categories = [self.products[p].get('category') for p in user_history if p in self.products]

        # Find candidates
        candidates = []
        for prod_id, prod_data in self.products.items():
            if prod_id not in user_history: # skip already purchased
                category = prod_data.get('category')

                # FAIRNESS: Score based on relevance + diversity
                if category in user_categories:
                    score = 0.5 # Bias toward user's interests
                else:
                    score = 0.5 # Explore new category

                # TRANSPARENCY: Explain why
                reason = f"Similar to your {category} purchases" if category in user_categories else f"Try new: {category}"

                candidates.append({
                    'product': prod_id,
                    'score': score,
                    'reason': reason
                })

        # Sort by score and return top N
        top_recs = sorted(candidates, key=lambda x: x['score'], reverse=True)[:num]

        return {
            'status': 'Success',
            'user_id': user_id,
            'history': user_history,
            'recommendations': top_recs
        }
```


Ensure logs do NOT store sensitive information such as passwords,

emails, or personal identifiers.

Add comments explaining ethical logging practices and privacy protection.

```
1 # An IDE
2 # ethical recommendation system
3 # ethical_logging.py
4
5 """Simple ethical logging for web applications"""
6
7 import logging
8 import re
9
10 class PrivacyFilter(logging.Filter):
11     """Remove sensitive data from logs"""
12
13     def filter(self, record):
14         """Mask passwords, emails, tokens, cards before logging"""
15         msg = record.getMessage()
16
17         # PRIVACY: Mask passwords
18         msg = re.sub('password=', 'password=***REDACTED***', msg, flags=re.IGNORECASE)
19
20         # PRIVACY: Mask emails (show domain only)
21         msg = re.sub('[-.\w+@]{1,25}@[a-z0-9]{1,40}', '[REDACTED]', msg)
22
23         # PRIVACY: Mask API keys and tokens
24         msg = re.sub('([a-zA-Z0-9]{32}|[a-zA-Z0-9]{40}|[a-zA-Z0-9]{64})', '[REDACTED]', msg, flags=re.IGNORECASE)
25
26         # PRIVACY: Mask credit cards (show last 4 digits)
27         msg = re.sub('^\d{15}(\d{4})$', '[REDACTED]', msg)
28
29         # PRIVACY: Mask phone numbers (show last 4 digits)
30         msg = re.sub('^\d{10}(\d{4})$', '[REDACTED]', msg)
31
32         record.msg = msg
33         return True
34
35 def setup_logger(name, log_filename):
36     """Set up logger with privacy protection"""
37     logger = logging.getLogger(name)
38     logger.setLevel(logging.INFO)
39
40     # Add privacy filter
41     privacy_filter = PrivacyFilter()
42
43     # Console handler
44     console_handler = logging.StreamHandler()
45     console_handler.addFilter(privacy_filter)
46     formatter = logging.Formatter('%(asctime)s - %(levelname)s - %(message)s')
47     console_handler.setFormatter(formatter)
48     logger.addHandler(console_handler)
49
50     # File handler
51     if log_filename:
52         file_handler = logging.FileHandler(log_filename)
53         file_handler.addFilter(privacy_filter)
54         file_handler.setFormatter(formatter)
55         logger.addHandler(file_handler)
56
57     # PRIVACY: Restrict file permissions (never read/write only)
58     import os
59     os.chmod(log_filename, 0o600)
60
61     return logger
62
63 def log_user_action(logger, action, user_id, **safe_details):
64     """Log user action with only safe fields"""
65     msg = f'ACTION: {action} | user: {user_id}'
66     if safe_details:
67         msg += ' | ' + ', '.join(f'{key}: {value}' for key, value in safe_details.items())
68     logger.info(msg)
69
70 # Example usage
71 if __name__ == '__main__':
72     print("--- Simple Ethical Logging Demo ---")
73
74     logger = setup_logger("app", log_filename="app.log")
75
76     print("Test 1: Password Masking")
77     logger.info("login with password=SecurePass123!")
78
79     print("Test 2: Email Masking")
80     logger.info("send email to user@example.com")
81
82     print("Test 3: API Key Masking")
83     logger.info("API key: sk_live_1234567890")
84
85     print("Test 4: Credit Card Masking")
86     logger.info("Payment with card 4112 1234 5678 9010")
87
88     print("Test 5: User Action Logging")
89     log_user_action(logger, "purchase", user_id=1, status="success", amount=99.99)
90
91     print("\n" + "=" * 40)
92     print("ETHICAL LOGGING PRACTICES")
93     print("=" * 40)
94
95     # 1. PRIVACY FILTER: Mask passwords, emails, tokens, cards
96     # 2. MINIMAL DATA: Only log necessary information
97     # 3. SECURE FILES: Set permissions to 060 (owner only)
98     # 4. CLEAR ACTIONS: Log for auditing and debugging
99     # 5. NO SENSITIVE: Never store sensitive data in logs
100
101     print("\n")
```

```
1 # An IDE
2 # ethical recommendation system
3 # ethical_logging.py
4
5 """Simple ethical logging for web applications"""
6
7 import logging
8 import re
9
10 class PrivacyFilter(logging.Filter):
11     """Remove sensitive data from logs"""
12
13     def filter(self, record):
14         """Mask passwords, emails, tokens, cards before logging"""
15         msg = record.getMessage()
16
17         # PRIVACY: Mask passwords
18         msg = re.sub('password=', 'password=***REDACTED***', msg, flags=re.IGNORECASE)
19
20         # PRIVACY: Mask emails (show domain only)
21         msg = re.sub('[-.\w+@]{1,25}@[a-z0-9]{1,40}', '[REDACTED]', msg)
22
23         # PRIVACY: Mask API keys and tokens
24         msg = re.sub('([a-zA-Z0-9]{32}|[a-zA-Z0-9]{40}|[a-zA-Z0-9]{64})', '[REDACTED]', msg, flags=re.IGNORECASE)
25
26         # PRIVACY: Mask credit cards (show last 4 digits)
27         msg = re.sub('^\d{15}(\d{4})$', '[REDACTED]', msg)
28
29         # PRIVACY: Mask phone numbers (show last 4 digits)
30         msg = re.sub('^\d{10}(\d{4})$', '[REDACTED]', msg)
31
32         record.msg = msg
33         return True
34
35 def setup_logger(name, log_filename):
36     """Set up logger with privacy protection"""
37     logger = logging.getLogger(name)
38     logger.setLevel(logging.INFO)
39
40     # Add privacy filter
41     privacy_filter = PrivacyFilter()
42
43     # Console handler
44     console_handler = logging.StreamHandler()
45     console_handler.addFilter(privacy_filter)
46     formatter = logging.Formatter('%(asctime)s - %(levelname)s - %(message)s')
47     console_handler.setFormatter(formatter)
48     logger.addHandler(console_handler)
49
50     # File handler
51     if log_filename:
52         file_handler = logging.FileHandler(log_filename)
53         file_handler.addFilter(privacy_filter)
54         file_handler.setFormatter(formatter)
55         logger.addHandler(file_handler)
56
57     # PRIVACY: Restrict file permissions (never read/write only)
58     import os
59     os.chmod(log_filename, 0o600)
60
61     return logger
62
63 def log_user_action(logger, action, user_id, **safe_details):
64     """Log user action with only safe fields"""
65     msg = f'ACTION: {action} | user: {user_id}'
66     if safe_details:
67         msg += ' | ' + ', '.join(f'{key}: {value}' for key, value in safe_details.items())
68     logger.info(msg)
69
70 # Example usage
71 if __name__ == '__main__':
72     print("--- Simple Ethical Logging Demo ---")
73
74     logger = setup_logger("app", log_filename="app.log")
75
76     print("Test 1: Password Masking")
77     logger.info("login with password=SecurePass123!")
78
79     print("Test 2: Email Masking")
80     logger.info("send email to user@example.com")
81
82     print("Test 3: API Key Masking")
83     logger.info("API key: sk_live_1234567890")
84
85     print("Test 4: Credit Card Masking")
86     logger.info("Payment with card 4112 1234 5678 9010")
87
88     print("Test 5: User Action Logging")
89     log_user_action(logger, "purchase", user_id=1, status="success", amount=99.99)
90
91     print("\n" + "=" * 40)
92     print("ETHICAL LOGGING PRACTICES")
93     print("=" * 40)
94
95     # 1. PRIVACY FILTER: Mask passwords, emails, tokens, cards
96     # 2. MINIMAL DATA: Only log necessary information
97     # 3. SECURE FILES: Set permissions to 060 (owner only)
98     # 4. CLEAR ACTIONS: Log for auditing and debugging
99     # 5. NO SENSITIVE: Never store sensitive data in logs
100
101     print("\n")
```

Expected Output #4:

- Logging code that avoids saving personal identifiers (e.g., passwords, emails), and includes comments about ethical logging practices.

```
Terminal - VS Code
Test 5: User Action Logging
2020-01-20 10:20:15,606 - app - INFO - ACTION: purchase | user: user_123 | {'status': 'success', 'amount': 99.99}

-----
ETHICAL LOGGING PRACTICES:
-----
1. PRIVACY FILTER: Mask passwords, email, tokens, cards
2. MINIMAL DATA: Only log necessary information
3. SECURE FILES: Set permissions to 600 (owner only)
4. USER ACTIONS: Log for auditing and debugging
5. NO SENSITIVE: Never store sensitive data in logs
2020-01-20 10:20:15,606 - app - INFO - ACTION: purchase | user: user_123 | {'status': 'success', 'amount': 99.99}

-----
ETHICAL LOGGING PRACTICES:
-----
1. PRIVACY FILTER: Mask passwords, email, tokens, cards
2. MINIMAL DATA: Only log necessary information
3. SECURE FILES: Set permissions to 600 (owner only)
4. USER ACTIONS: Log for auditing and debugging
5. NO SENSITIVE: Never store sensitive data in logs

-----
ETHICAL LOGGING PRACTICES:
-----
1. PRIVACY FILTER: Mask passwords, email, tokens, cards
2. MINIMAL DATA: Only log necessary information
3. SECURE FILES: Set permissions to 600 (owner only)
4. USER ACTIONS: Log for auditing and debugging
5. NO SENSITIVE: Never store sensitive data in logs
```

Task Description #5:

- Ask Copilot to generate a machine learning model.

Then, prompt

it to add documentation on how to use the model responsibly (e.g., explainability, accuracy limits).

PROMPT: Generate a Python machine learning model (including data loading, training, and prediction steps).

Add inline documentation or a README-style comment section explaining how to use the model responsibly, including accuracy limitations, explainability considerations, fairness concerns, and appropriate use cases and restrictions.

